Activation Functions: Sigmoid, Tanh and ReLU

Contents

1. Overview . . . 2
2. Prerequisites . . . 3
3. What an activation function is . . . 3
4. Why a network needs a non-linear activation . . . 4
    4.1 An experiment: linear activations on circles . . . 4
    4.2 The algebra: linear layers collapse . . . 5
5. Properties of an ideal activation function . . . 6
    5.1 Non-linear . . . 6
    5.2 Differentiable . . . 6
    5.3 Computationally inexpensive . . . 7
    5.4 Zero-centred . . . 7
    5.5 Non-saturating . . . 9
6. Sigmoid . . . 10
    6.1 Shape and slope . . . 10
    6.2 Advantages . . . 10
    6.3 Disadvantages . . . 10
7. Tanh . . . 12
    7.1 Shape and slope . . . 12
    7.2 Advantages and disadvantages . . . 13
8. ReLU . . . 14
    8.1 Advantages . . . 15
    8.2 Disadvantages . . . 17
9. Summary . . . 18
10. Sources . . . 18
11. Key terms . . . 19

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Sigmoid function; Perceptron; Multi-layer perceptron (MLP); Vanishing gradient; Improving a neural network.
- **Leads to:** Dying ReLU problem; GELU activation; Xavier and He initialisation; Recurrent neural network (RNN).
- **Compare with:** Sigmoid function; Leaky ReLU, PReLU, ELU and SELU.

1. Overview

Key point: An activation function is the non-linear step inside every node. Without it a deep network is only a linear model. Sigmoid and tanh squash their input and slow deep networks down; **ReLU** (G-1668) does not squash positive values and is the default for hidden layers today.

Every node of a neural network computes a **weighted sum** (G-2119) z of its inputs plus a **bias** (G-287), then passes z through a function. This function, the **activation function** (G-165), decides how strongly the node "fires". This Note answers three questions:

- **Why** a network needs a non-linear activation at all.
- **What** makes an activation function good: five properties.
- **How** the three classic functions, **sigmoid** (G-1798), tanh and ReLU, score on those properties.

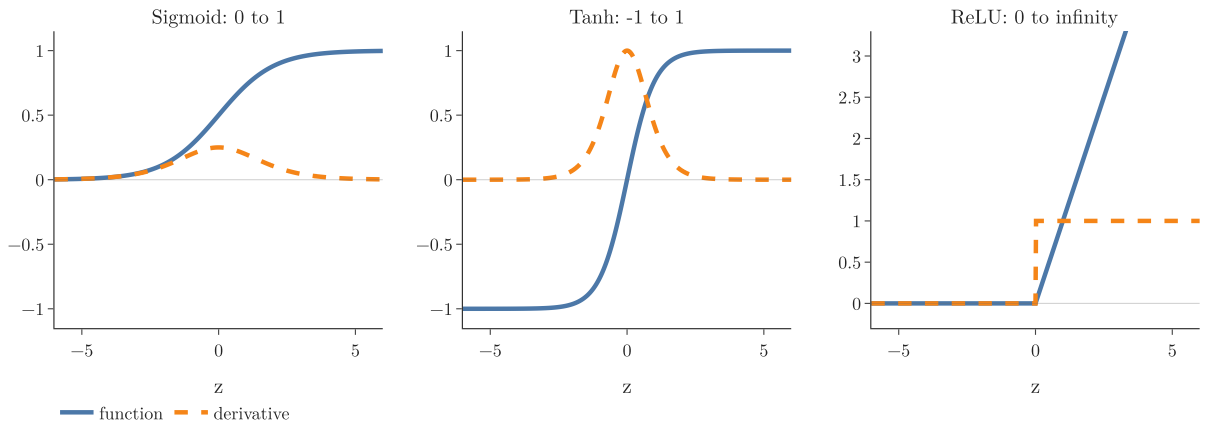


Figure 1: Sigmoid, tanh and ReLU (solid) with their derivatives (dashed). Sigmoid's slope is never above 0.25, tanh's never above 1; both fall to 0 for large $|z|$. ReLU's slope is 0 or 1

Figure 1 shows the three functions side by side. Most of this Note is about what their shapes, and the shapes of their slopes, do to training.

2. Prerequisites

- The perceptron: weighted sum, bias and **activation function** (G-165).
- The sigmoid and the sigmoid derivative. The **sigmoid** (G-1798) and its slope:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq 0.25$$

- The vanishing gradient problem: why small slopes stop early layers from learning.
- Backpropagation in code: **gradients** (G-863) as chain-rule products.

3. What an activation function is

> **Key point:** A node computes $z = \sum w_i x_i + b$ and outputs $a = g(z)$. The function g is the activation function: a gate deciding whether, and how much, the node is activated.

Take a network with two inputs, one hidden layer of two nodes and one output node. The first hidden node computes the **weighted sum** (G-2119) of its inputs plus its bias:

$$z = w_1 x_1 + w_2 x_2 + b_1$$

The node then outputs $a = g(z)$. The function g is the **activation function** (G-165), also called the **transfer function** (G-2004). With n inputs the output is

$$a = g\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

where $\sum$ means "add up the terms for $i = 1$ to n".

A worked case: weights $w_1 = 2$ and $w_2 = -1$, **bias** (G-287) $b_1 = 0.5$, inputs $x_1 = 3$ and $x_2 = 1$, and a sigmoid for g:

$$z = 2 \times 3 + (-1) \times 1 + 0.5 = 5.5$$

$$a = g(5.5) = \frac{1}{1 + e^{-5.5}} = 0.996$$

We can picture g as a gate between what flows into a node and what flows out. The gate decides whether the node is activated and, if so, how strongly. The **sigmoid** (G-1798) and the softmax (G-1830) are two activation functions already met; this Note adds **tanh** (G-1947) and **ReLU** (G-1668).

4. Why a network needs a non-linear activation

> **Key point:** Without a non-linear activation, any number of layers collapses into one linear layer: the network's **decision boundary** (G-555) (the line between the regions given to each class) is a straight line, however deep it is.

4.1 An experiment: linear activations on circles

> **Key point:** Two hidden layers of 32 nodes with linear activations reach 50% accuracy on two circles, the same as guessing. With **ReLU** (G-1668) the same network reaches 100%.

The data is `make_circles` (G-110) from scikit-learn. Each **observation** (G-1374; one record, one row of the data table) has:

- two **features** (G-772; input variables) x_1 and x_2, its position on the plane;
- a **target** (G-1949; the output we predict): class 1 for the inner ring, class 0 for the outer ring.

There are 300 observations, 150 per class. No straight line separates the two rings. We train the same network twice for 200 **epochs** (G-696; one epoch is one pass over all the training data):

- two hidden layers of 32 nodes with `activation="linear"`, which means $g(z) = z$, the same as no activation at all;
- the same two layers with `activation="relu"`.

The output node is a **sigmoid** (G-1798) in both, because this is binary classification.

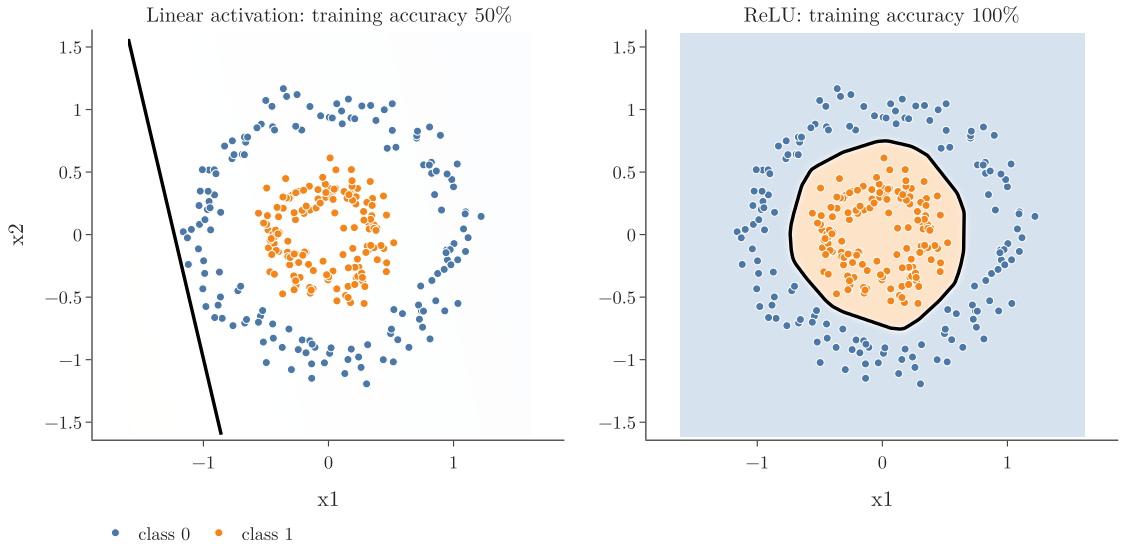


Figure 2: Decision regions after 200 epochs. Left: with linear activations the boundary (black) is a straight line, here pushed outside the data; every point gets about 0.5. Right: with ReLU the boundary bends around the inner ring

In Figure 2, the background shade is the probability the network gives to class 1 at that spot: light orange where it favours class 1, light blue where it favours class 0, white at 0.5. The black line is the decision boundary, where the probability is 0.5. The dots are the training points (blue: class 0, orange: class 1).

Figure 2 shows the result. The linear network ends with a loss of 0.694 and 50% training accuracy, which is exactly guessing: its **decision boundary** (G-555), the line between the regions given to each class, can only be straight, and the best straight one is useless here. The ReLU network ends at 100%, with a decision boundary that bends around the inner ring.

> **Python:** The only difference between the two models is one argument.
> ```
> keras.layers.Dense(32, activation="linear")   # no bend
> keras.layers.Dense(32, activation="relu")     # bends
> ```

4.2 The algebra: linear layers collapse

> **Key point:** Two linear layers $W_2(W_1 a_0 + b_1) + b_2$ equal one layer $W' a_0 + b'$. The output is a degree-1 function of the input.

Stacked matrices without an activation multiply into a single matrix (see matrix multiplication). The **bias** (G-287) terms add only a shift, so they do not change that.

1. **In words:** the first layer computes $a_1 = W_1 a_0 + b_1$; the second feeds that into $a_2 = W_2 a_1 + b_2$; multiplying out gives one weight matrix and one bias.

2. **Formula:**

$$a_2 = W_2(W_1 a_0 + b_1) + b_2 = \underbrace{W_2 W_1}_{W'}\, a_0 + \underbrace{W_2 b_1 + b_2}_{b'}$$

3. **Example:** with single numbers $W_1 = 2$, $b_1 = 1$, $W_2 = 3$, $b_2 = -1$:

$$a_2 = 3(2x + 1) - 1 = 6x + 2$$

Two layers, and the result is still a straight line in x.

The Notebook checks the collapse on the trained linear network of Figure 2. The network's three weight matrices multiply into one 2×1 matrix W', and one layer with W' and b' gives the same predictions to within 6×10^{-8}. A network with linear activations is therefore linear regression (G-1094; regression output) or logistic regression (G-1120; sigmoid output), however many layers it has.

With a non-linear g, such as the **sigmoid** (G-1798), $g(W_1 a_0 + b_1)$ is no longer a degree-1 expression. Feeding it into the next layer gives a relationship between input and output that is not a polynomial of degree 1, so the network can follow non-linear patterns. Capturing non-linear patterns is the whole reason **activation functions** (G-165) exist.

5. Properties of an ideal activation function

> **Key point:** A good **activation function** (G-165) is non-linear, differentiable, cheap to compute, zero-centred and non-saturating. No classic function has all five.

5.1 Non-linear

> **Key point:** Non-linearity is what lets a network capture non-linear patterns. Non-linearity is the one property that cannot be dropped.

The relationship between z and $g(z)$ must not be a straight line, as section 4 showed. With a non-linear activation, the universal approximation theorem (G-2052) says that enough nodes and layers can approximate any pattern in the data.

5.2 Differentiable

> **Key point:** Gradient descent needs the derivative of the activation at every step, so it must exist.

Backpropagation (G-247) multiplies the activation's **derivative** (G-595) into every **gradient** (G-863) (see backpropagation in code). Without a derivative there is no gradient, no update and no training.

The property is important but not strict. **ReLU** (G-1668) has no derivative at exactly $z = 0$ and is still the most used activation (section 8.2 explains how this is handled).

5.3 Computationally inexpensive

> **Key point:** The function and its **derivative** (G-595) are computed for every node, every observation and every epoch; a slow function makes training slow.

A network with a million nodes trained on a million observations for 100 epochs evaluates its activation this many times:

$$10^6 \text{ nodes} \times 10^6 \text{ observations}$$

$$= 10^{12} \text{ per epoch}$$

$$10^{12} \times 100 \text{ epochs} = 10^{14}$$

Functions built from a simple comparison are faster than those built from exponentials.

5.4 Zero-centred

> **Key point:** Outputs with mean about 0, some positive and some negative, act like normalised inputs to the next layer and make training converge faster.

A **zero-centred activation function** (G-165) (G-2148) gives outputs whose mean over a layer is around 0. Training converges faster when the inputs are normalised (see **standardization** (G-1874)), and the outputs of one layer are the inputs of the next. **Tanh** (G-1947) is zero-centred; section 6.3 shows what goes wrong when an activation is not.

Figures 3 and 4 show the effect on the simplest case: one node $\hat{y} = w_1 a_1 + w_2 a_2$ learning, one observation at a time, a target that needs $w_1 = 1$ and $w_2 = -1$. The inputs a_1, a_2 are 200 random values between 0.1 and 0.9, like **sigmoid** (G-1798) outputs (left), or the same values minus 0.5, centred on 0 like tanh outputs (right). Both start from $(-1, 1)$ with the same **learning rate** (the size of each step), 2.5.

The loss of this node is a surface over the two weights. For a pair (w_1, w_2) the height is the mean squared error over the 200 observations, drawn as $\log_{10}(L + 0.001)$ so that the region near the minimum is visible. Two points on it, for either kind of input:

$$L(-1, 1) = 0.50 \quad \text{(the start)}$$

$$L(1, -1) = 0 \quad \text{(the target)}$$

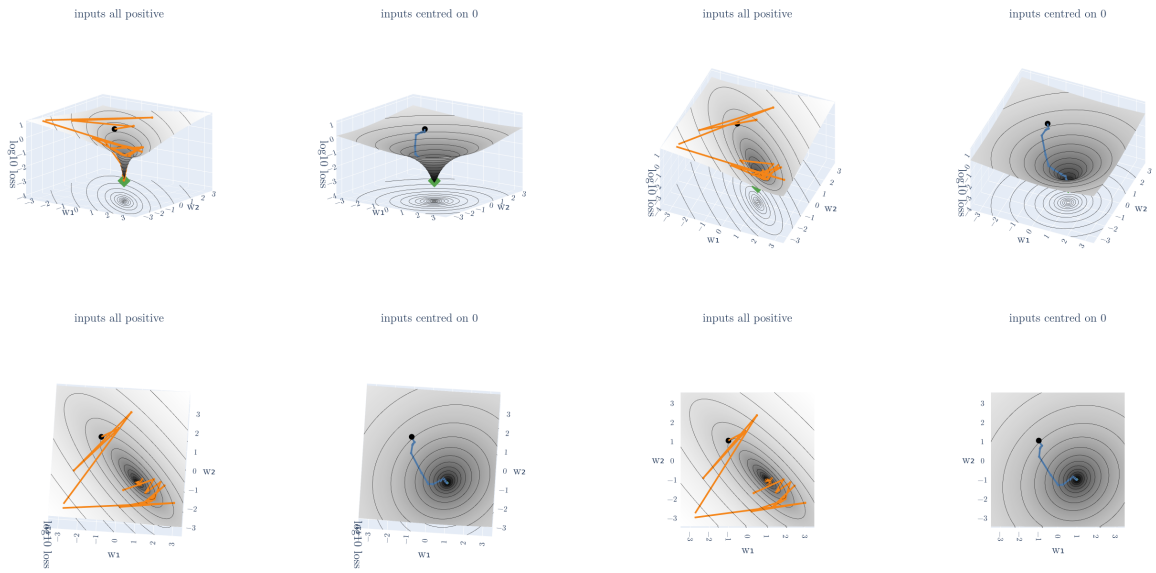


Figure 3: The loss surfaces of the one-node example, for all-positive inputs (left) and inputs centred on 0 (right). The camera tilts from a side view to the top view; the black lines join points at the same height. Black dot: start; green diamond: the target; coloured line: the 40 steps of Figure 4

Figure 3 shows the two surfaces: both are bowls with the target at the bottom. The left bowl is a long, narrow valley tilted across the weights; the right bowl is round. The contour map in Figure 4 (grey lines) is each bowl seen from above: each line joins points at the same loss; lines close together mean a steep slope, and the centre is the target.

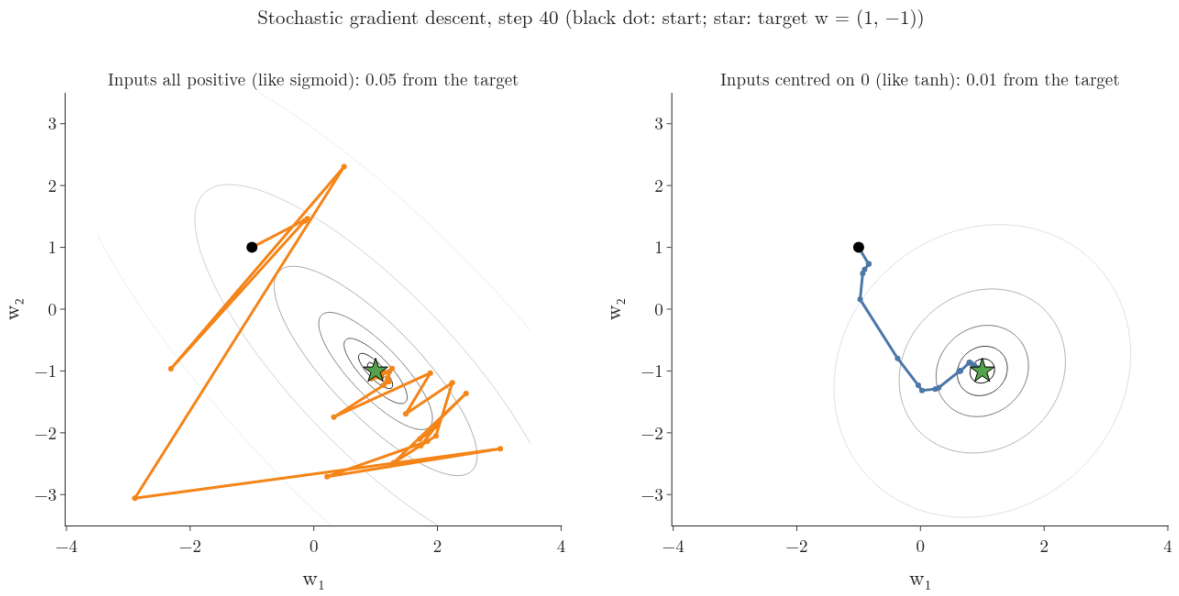


Figure 4: Stochastic gradient descent on one node with two weights. Left: inputs all positive. Right: the same inputs centred on 0. Grey: contours of the loss; black dot: start; star: the target weights.

1. **Each step's direction.** Here y is the target and $\hat{y}$ the node's output. The **gradient** (G-863) of the squared error has one part for each weight:

$$\frac{\partial L}{\partial w_1} = (\hat{y} - y)\, a_1$$

$$\frac{\partial L}{\partial w_2} = (\hat{y} - y)\, a_2$$

   With positive inputs, both parts have the sign of $\hat{y} - y$, so every orange step moves both weights up or both down: along a diagonal.
2. **The valley.** With positive inputs the loss forms a long, narrow valley (left contours). Steps along the diagonal keep overshooting across it.
3. **The result.** The title of Figure 4 counts the steps. After 20 steps the positive-input node is still 1.01 away from the target; the centred one is 0.10 away. Centred inputs make the loss contours round, so the steps head straight for the target (LeCun et al. 1998, §4.3).

5.5 Non-saturating

> **Key point:** A saturating function squeezes any input into a fixed range, so its slope goes to 0 at the ends. The flat ends cause the vanishing gradient problem.

A **saturating function** (G-1741) squeezes an input from $-\infty$ to ∞ into a bounded range and flattens out there. The **sigmoid** (G-1798) squeezes into 0 to 1, tanh into -1 to 1. A **non-saturating function** (G-1339) has no such ceiling: **ReLU** (G-1668), $\max(0, z)$, grows as large as its input.

Where a function is flat its slope is near 0, and a product of many near-zero slopes is the vanishing gradient (G-2070). The vanishing gradient appears with saturating activations, which is why non-saturation matters.

6. Sigmoid

> **Key point:** The **sigmoid** (G-1798) gives a probability, is non-linear and differentiable; but it saturates, is not zero-centred and needs an exponential. Today it is used mainly in the output layer for binary classification.

6.1 Shape and slope

> **Key point:** Output 0.5 at $z = 0$, near 0 or 1 for large $|z|$. Its **derivative** (G-595) peaks at 0.25 and is almost 0 outside -3 to 3.

The sigmoid is (Figure 1, left)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

For a large z it is close to 1, for a very negative z close to 0, and $\sigma(0) = 0.5$.

Its derivative $\sigma(z)(1 - \sigma(z))$ is largest at $z = 0$, where it is 0.25. Between -3 and 3 it is reasonably large; outside, it is close to 0. At $z = 3$ it is 0.045 and at $z = 5$ only 0.0066.

6.2 Advantages

> **Key point:** Output reads as a probability; non-linear; differentiable everywhere.

1. **Probability output.** The output lies between 0 and 1, so it can be read as a probability. For this reason we put a **sigmoid** (G-1798) in the output node for binary classification: near 1 means "yes", near 0 means "no".
2. **Non-linear.** The exponential makes it non-linear, so it can capture non-linear patterns.
3. **Differentiable everywhere,** with a **derivative** (G-595) that is easy to write in terms of the function itself.

6.3 Disadvantages

> **Key point:** Saturation causes the **vanishing gradient** (G-2070); all-positive outputs force every weight into a node to move in the same direction; the exponential is slow.

1. Saturating, so the gradient vanishes. Gradient descent updates each weight with the learning rate η (the step size):

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

If z is outside about -3 to 3, the **sigmoid** (G-1798)'s slope is near 0, the gradient is near 0 and $w_{\text{new}} \approx w_{\text{old}}$: no training. In a deep network these small slopes multiply from layer to layer (see the vanishing gradient problem).

z becomes large in two ways:

- **Large inputs.** Usually not the cause, because we scale the inputs first.
- **Large weights and biases.** If the starting weights are big, z is big and the node starts saturated (see weight initialisation).

Saturation is the main reason the sigmoid has disappeared from hidden layers (Goodfellow et al. 2016, §6.3.2).

2. Not zero-centred, so training is slow. Every sigmoid output is positive. Take two sigmoid nodes, with outputs a_{21} and a_{22}, feeding one output node through weights w_{21} and w_{22}:

$$z_{31} = w_{21}\, a_{21} + w_{22}\, a_{22} + b_{31}$$

1. **In words:** by the **chain rule** (G-371), the **gradient** (G-863) of each weight is a factor shared by both, times that weight's input.
2. **Formula:** with the shared factor (called δ) equal to the product of two slopes,

$$\delta = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_{31}}$$

$$\frac{\partial L}{\partial w_{21}} = \delta \cdot a_{21}$$

$$\frac{\partial L}{\partial w_{22}} = \delta \cdot a_{22}$$

3. **Example:** with $\delta = -0.5$ and sigmoid outputs $a_{21} = 0.6$ and $a_{22} = 0.3$:

$$\frac{\partial L}{\partial w_{21}} = -0.5 \times 0.6 = -0.30$$

$$\frac{\partial L}{\partial w_{22}} = -0.5 \times 0.3 = -0.15$$

   Both are negative. Had a_{22} been a **tanh** (G-1947) output of -0.3, the second would be positive:

$$\frac{\partial L}{\partial w_{22}} = -0.5 \times (-0.3) = +0.15$$

Because a_{21} and a_{22} are both positive, both gradients take the sign of δ. So in one update the weights into a node either all increase or all decrease; they can never move in opposite directions.

(a) Directions one update can take **(b) Reaching the best weights**

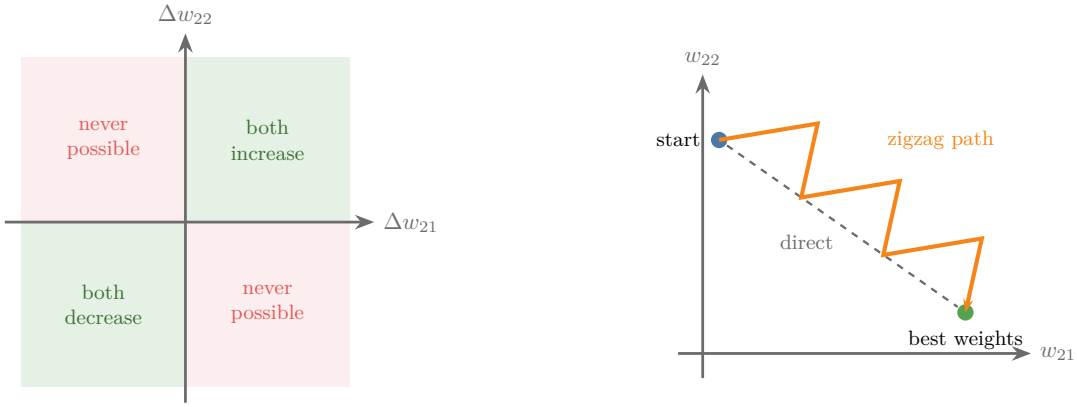


Figure 5: (a) With all-positive inputs, one update moves every weight into a node the same way: only the green quadrants are possible. (b) When the best weights need one weight up and the other down, gradient descent has to zigzag

Figure 5 shows the consequence. If reaching the minimum needs w_{21} to grow while w_{22} shrinks, no single step can do that. Gradient descent zigzags, like driving to a place on our left when we are not allowed to turn left, and it converges slowly (LeCun et al. 1998).

The Notebook checks the sign rule with one hidden layer of 10 nodes feeding a sigmoid output, for one observation. With sigmoid hidden nodes, the 10 gradients of the output node's weights all have the same sign (++++++++++). With tanh hidden nodes they are mixed (−+−++++++−+).

3. Computationally expensive. Each evaluation needs an exponential. In the Notebook, 10 million sigmoids take about 3,200 ms, against about 800 ms for **ReLU** (G-1668): about four times as long. The exact times depend on the computer.

In short, the sigmoid is rarely used in hidden layers today. The sigmoid stays in the output layer for binary classification.

7. Tanh

> **Key point:** Tanh is a stretched **sigmoid** (G-1798) with range −1 to 1. **Tanh** (G-1947) is zero-centred, which fixes the sigmoid's slow training, but it still saturates and still needs exponentials.

7.1 Shape and slope

> **Key point:** $\tanh(z) = (e^z - e^{-z})/(e^z + e^{-z})$, with **derivative** (G-595) $1 - \tanh^2(z)$, peaking at 1.

1. **In words:** the **hyperbolic tangent (tanh)** is an S-shaped curve like the **sigmoid** (G-1798), but from -1 to 1, passing through 0 at $z = 0$.
2. **Formula:**
$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$
$$\tanh'(z) = 1 - \tanh^2(z)$$
3. **Example:** at $z = 1$, $e^1 = 2.718$ and $e^{-1} = 0.368$, so
$$\tanh(1) = \frac{2.718 - 0.368}{2.718 + 0.368}$$
$$= \frac{2.350}{3.086} = 0.762$$
$$\tanh'(1) = 1 - 0.762^2 = 0.42$$

Like the sigmoid, its derivative is written in terms of the function itself. The derivative's peak is 1 instead of 0.25, and the curve is steeper (Figure 1, middle).

> **Extra: Tanh** (G-1947) is a rescaled sigmoid: it equals twice the sigmoid of $2z$, minus 1. To see it, start from the right side and multiply the top and bottom of the last fraction by e^z:
> $$2\sigma(2z) - 1 = \frac{2}{1 + e^{-2z}} - 1$$
> $$= \frac{1 - e^{-2z}}{1 + e^{-2z}}$$
> $$= \frac{e^z - e^{-z}}{e^z + e^{-z}} = \tanh(z)$$
> The Notebook confirms the identity numerically. The steeper middle of tanh also means tanh flattens sooner: at $z = 3$ its slope is 0.0099, against the sigmoid's 0.045.

7.2 Advantages and disadvantages

> **Key point:** Gains zero-centred outputs; keeps saturation and exponentials.

Advantages:

1. **Non-linear**, so it captures non-linear patterns.
2. **Differentiable** everywhere.
3. **Zero-centred.** Outputs are both positive and negative. The gradients of the weights into a node can then have different signs, and training is faster than with the **sigmoid** (G-1798) (LeCun et al. 1998, §4.4). In the Notebook, over all 300 observations, a 128-node tanh layer has mean output 0.000 with half the values positive, while a sigmoid layer has mean 0.500 with every value positive.

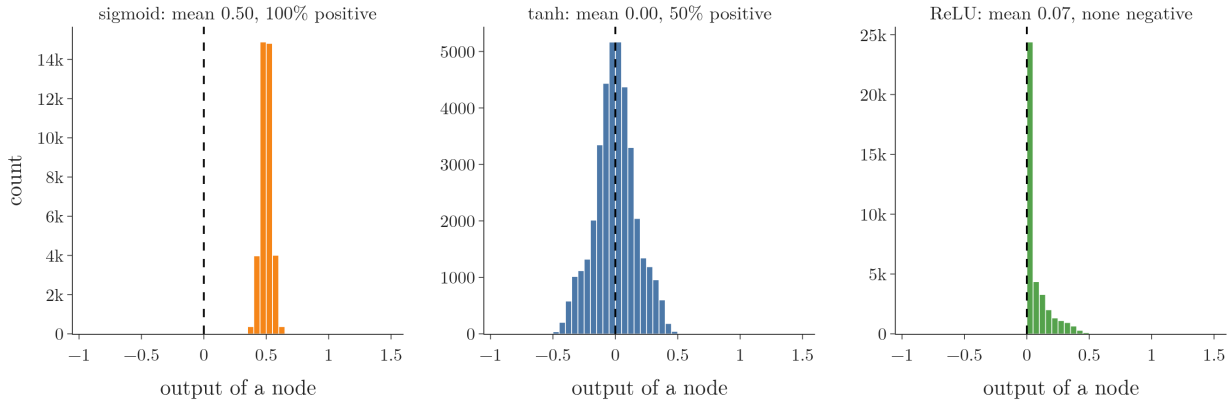


Figure 6: The outputs of one freshly initialised 128-node layer for the 300 circles observations, with three activations: 38,400 values each. Dashed line: 0.

Figure 6 shows those layer outputs. The sigmoid's values all sit above 0, around 0.5. The tanh values spread evenly on both sides of 0, so the next layer receives inputs centred on 0, as in the right half of Figure 4. **ReLU** (G-1668)'s values are 0 or positive, never negative: ReLU is not zero-centred either (section 8.2).

Disadvantages:

1. **Saturating.** For large $|z|$ the slope is 0, so **tanh** (G-1947) also suffers from the vanishing gradient.
2. **Computationally expensive**, because of the exponentials (about 1,250 ms for 10 million values in the Notebook). Tanh comes out faster than the sigmoid here only because of how the Notebook computes them: the sigmoid is written as three separate NumPy steps (an exponential, an addition, a division), while `np.tanh` is one built-in step. Both are slower than ReLU, which needs no exponential.

Tanh fixed the sigmoid's slow training from non-zero-centred outputs, but not the **vanishing gradient** (G-2070).

8. ReLU

> **Key point: ReLU** (G-1668), $\max(0, z)$, is non-linear, does not saturate for positive z, is cheap and converges faster than **sigmoid** (G-1798) and tanh. Its weaknesses: no derivative at 0, not zero-centred, and dying nodes.

ReLU (rectified linear unit) is $\max(0, z)$: 0 for negative z, z itself for positive z (Figure 1, right). ReLU is the most used activation in hidden layers today, the recommended default (Goodfellow et al. 2016, §6.3).

8.1 Advantages

> **Key point:** Non-linear (one **ReLU** (G-1668) has a corner; two make a bump), non-saturating on the positive side, no exponential, faster convergence.

1. Non-linear. ReLU looks like two straight segments, which makes some people think it is linear. A linear function would be $f(z) = z$ everywhere; $\max(0, z)$ has a corner, so it is not. Combining ReLUs builds more complex shapes:

1. **In words:** subtract one shifted ReLU from another.
2. **Formula:**
$$f(x) = \max(0, x + 1) - \max(0, x - 1)$$
3. **Example:**
$$f(-2) = 0 - 0 = 0$$
$$f(0) = 1 - 0 = 1$$
$$f(3) = 4 - 2 = 2$$

   Flat, then rising, then flat again.

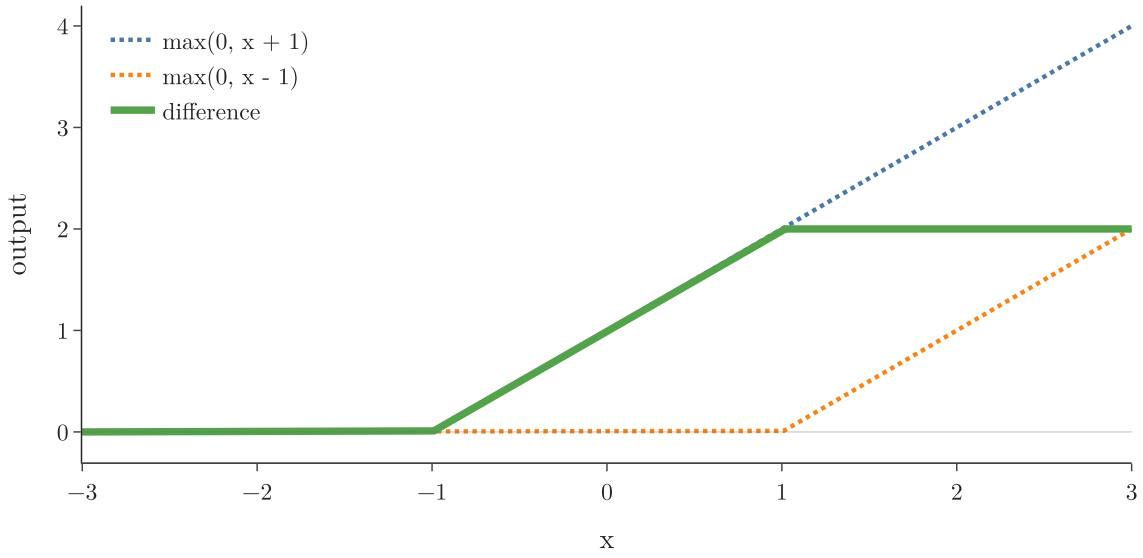


Figure 7: Two ReLUs (dotted) and their difference (green): flat, rising, flat. No straight line has two corners

Figure 7 shows the result: a curve with two corners. The ReLU **decision boundary** (G-555) in Figure 2 is built the same way, from straight segments joined at corners: it is **piecewise linear** (G-1496).

How the bends fit data. A network makes the same move with its own weights. Take a drug that works at a medium dosage and fails at a low and a high one: three observations,

dosage 0, 0.5 and 1, with targets 0, 1 and 0. A network with two ReLU hidden nodes and a final ReLU on the output node fits them in six steps, one per frame of Figure 8:

1. **Node 1** computes $\max(0,\ 1.70 \times \text{dosage} - 0.85)$: flat at 0 up to dosage 0.5, then a rising line.
2. **Its output weight,** -40.8**,** flips the bent line and stretches it downwards.
3. **Node 2,** times its output weight, gives the straight line $34.0 \times$ dosage.
4. **The output node adds the two:** a wedge with its corner at dosage 0.5.
5. **The output bias,** -16**,** moves the wedge down until only its tip is above 0.
6. **The final ReLU** sets every negative value to 0. A narrow peak is left.

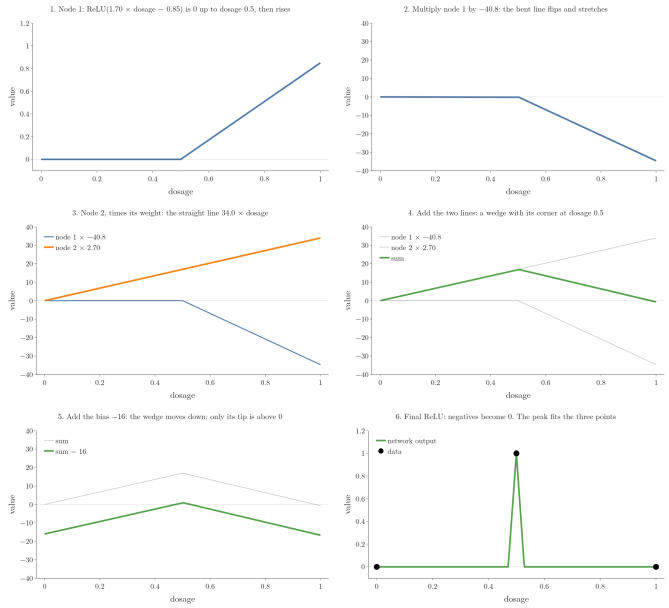


Figure 8: Two ReLU nodes fitting three points, one operation per frame: the bent line of node 1, its output weight, the straight line of node 2, their sum, the bias, the final ReLU. Numbers after StatQuest, "Neural Networks Pt. 3: ReLU In Action!!!"

Check the three observations:

Dosage	Node 1 $\times\ (-40.8)$	Node 2 line	Sum -16	After the final ReLU
0	0	0	-16	0
0.5	0	17.0	1.0	1.0
1	$0.85 \times (-40.8) = -34.7$	34.0	-16.7	0

Each ReLU node contributes one bent line. The weights and **biases** (G-287) slide, flip and stretch these lines, and their sum is a new shape with one corner per node. More nodes give more corners, so the network can follow more complicated patterns.

2. Non-saturating in the positive region. For positive z the output grows without limit and the slope stays 1, so the **gradient** (G-863) does not vanish there.

3. Computationally inexpensive. Computing it is one comparison, and its **derivative** (G-595) is 0 or 1. No exponentials.

4. Faster convergence than **sigmoid** (G-1798) and tanh. On the CIFAR-10 images, a four-layer ReLU network reached a 25% training error about six times faster than the same network with tanh (Krizhevsky et al. 2012, §3.1, Figure 1).

8.2 Disadvantages

> **Key point:** No **derivative** (G-595) at exactly 0 (we just pick one), outputs never negative (batch normalisation helps), and nodes that can die.

1. Not differentiable at $z = 0$. The two segments meet at a corner, which has no single slope. In code we simply choose one: slope 0 for $z < 0$, slope 1 for $z > 0$, and one of the two at exactly 0.

2. Not zero-centred. Like the sigmoid's, **ReLU** (G-1668)'s outputs are never negative, with the same effect on the gradient signs as in section 6.3. Batch normalisation (G-266) addresses this: it normalises the values passed between layers.

3. The dying ReLU problem (G-650). A node whose z stays negative outputs 0 with slope 0, so it stops learning (see using ReLU against vanishing gradients). The causes and the fixes are in the ReLU variants. GELU and SiLU are smooth versions of ReLU used in transformers.

> **Python:** Choosing activations in Keras.

```
model = keras.Sequential([
    keras.Input(shape=(2,)),
    keras.layers.Dense(32, activation="relu"),  # hidden: ReLU
    keras.layers.Dense(32, activation="tanh"),  # or tanh
    keras.layers.Dense(1, activation="sigmoid")])  # output
```

> The strings `"sigmoid"`, `"tanh"`, `"relu"` and `"linear"` pick the function for every node of that layer.

9. Summary

Property	Sigmoid	Tanh	ReLU
Formula	$1/(1+e^{-z})$	$(e^z - e^{-z})/(e^z + e^{-z})$	$\max(0, z)$
Range	0 to 1	-1 to 1	0 to ∞
Largest slope	0.25	1	1
Non-linear	yes	yes	yes
Differentiable	everywhere	everywhere	except at 0
Zero-centred	no	yes	no
Saturating	yes	yes	not for $z > 0$
Cost (10 million values, our computer)	about 3,200 ms	about 1,250 ms	about 800 ms
Used today	output, binary classification	some hidden layers	most hidden layers

- Without a non-linear activation, any network is a linear model (50% vs 100% on the circles), because any number of linear layers collapses into one linear layer.
- An ideal activation is non-linear, differentiable, cheap, zero-centred and non-saturating, so these five properties are the checklist for judging any activation.
- **Saturating functions** (G-1741) (sigmoid, tanh) cause the vanishing gradient.
- All-positive outputs (sigmoid, **ReLU** (G-1668)) make the weights into a node move together, so training zigzags.
- ReLU is the default for hidden layers, because it is cheap and does not saturate for $z > 0$; its dying-node problem leads to the ReLU variants.

10. Sources

Built from

- CampusX, "Activation Functions in Deep Learning | Sigmoid, Tanh and Relu Activation Function", YouTube, https://www.youtube.com/watch?v=7LcUkgzx3AY
- StatQuest with Josh Starmer, "Neural Networks Pt. 3: ReLU In Action!!!", YouTube, https://www.youtube.com/watch?v=68BZ5f7P94E (the dosage example of section 8.1: two ReLU nodes building a shape, Figure 8)

Other references

- LeCun, Y., Bottou, L., Orr, G. B. and Müller, K.-R. (1998). Efficient BackProp. In *Neural Networks: Tricks of the Trade*, Springer, section 4.3 (all-positive inputs make the weight updates zigzag) and section 4.4 (symmetric sigmoids such as tanh converge faster than the logistic sigmoid).
- Krizhevsky, A., Sutskever, I. and Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. NeurIPS, section 3.1 (ReLU trains several times faster than tanh).

- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*, MIT Press, section 6.3 (ReLU as the default hidden unit; sigmoid hidden units discouraged because they saturate).

11. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
ReLU (G-1668)	Rectified linear unit, the activation $\max(0, z)$: it passes a positive input unchanged and turns a negative one into 0; the usual default activation for hidden layers.
Transfer function (G-2004)	Another name for the activation function: the function a node applies to its weighted sum to give its output.
Tanh (hyperbolic tangent) (G-1947)	An activation function that squashes any number into an S-curve from -1 to 1, centred on 0: $\tanh(z) = (e^z - e^{-z})/(e^z + e^{-z})$, with derivative $1 - \tanh^2(z)$.
`make_circles` (G-110)	scikit-learn function that generates two concentric rings of observations, one ring per class.
Zero-centred activation (G-2148)	An activation whose outputs average about 0 over a layer, positive and negative; it feeds the next layer roughly centred inputs, which helps training converge faster.
Saturating function (G-1741)	A function that squeezes any input into a bounded range, such as the sigmoid into 0 to 1, so its slope goes to 0 at the ends; those near-zero slopes make gradients vanish in deep networks.
Non-saturating function (G-1339)	A function with no ceiling on its output, so it does not flatten out, such as ReLU for positive inputs; its slope does not shrink to 0, which avoids the vanishing gradient that saturating functions cause.
Weighted sum (G-2119)	In self-attention, the value vectors multiplied by their weights and added together; the result is the word's new, context-aware vector.
Bias (of a model) (G-287)	Error from a model being too simple to capture the true relationship.

Term	Meaning
Activation function (G-165)	The function that turns a node's weighted sum z into its output, bringing it into a fixed range.
Sigmoid function (G-1798)	An S-shaped function that squashes any number into the range 0 to 1, $\sigma(z) = 1/(1 + e^{-z})$; it turns a score into a probability, as in logistic regression.
Gradient (of the loss) (G-863)	How fast the loss changes with each weight and bias, collected in one vector (the partial derivatives of the loss); it points in the direction in which the loss rises fastest.
Softmax function (G-1830)	A function that turns a list of scores into probabilities that add up to 1: raise e to each score and divide by the total, $e^{z_k} / \sum_j e^{z_j}$; it gives one probability per class in multi-class models.
Decision boundary (G-555)	A line or curve that separates the classes in classification.
Observation (G-1374)	One record of a dataset: one row of the data table.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
Epoch (G-696)	One full update of the parameters using the whole training set.
Linear regression (G-1094)	A model that predicts a number from the inputs by fitting the straight line (or, with several inputs, the plane) that runs closest to all the training points.
Logistic regression (G-1120)	A classification algorithm that passes a weighted sum of the inputs through the sigmoid to give the probability of a class; the line (or plane) where that probability is 0.5 is the boundary it learns between the classes.
Universal approximation theorem (G-2052)	A network with a hidden layer and enough neurons can approximate any continuous function.
Backpropagation (G-247)	The algorithm that computes the gradient of the loss with respect to every weight and bias by applying the chain rule backward from the output; gradient descent then uses these gradients to train the network.

Term	Meaning
Derivative (G-595)	The slope of a function at a point: how fast the output changes as the input changes there. It is zero at the lowest point of a smooth curve, which is how training finds a minimum.
Standardization (G-1874)	Scaling a column to mean 0 and standard deviation 1 (subtract the mean, divide by the standard deviation), so features measured on different scales become comparable.
Vanishing gradient (G-2070)	Gradients shrinking towards 0 as they pass through many layers, which slows learning.
Chain rule (G-371)	To differentiate a function of a function, multiply the outer derivative by the inner derivative.
Piecewise linear (G-1496)	Made of straight segments joined at bends; the shape of a ReLU network's decision boundary.
Batch normalisation (G-266)	A layer placed between layers of a network that re-centres and re-scales its inputs during training; this keeps the activations away from the flat ends of sigmoid and tanh, which helps against vanishing gradients.
Dying ReLU problem (G-650)	ReLU nodes ending up with a negative weighted sum for every input, so they output 0 and stop learning.
Linear activation (G-1089)	An activation that passes a node's weighted sum through unchanged, $f(z) = z$. Used in the output layer for regression.